組織編集ダイアログUI設計書

メタデータ

title: 組織編集ダイアログUI設計書
category: UI設計
version: 1.0.0
last_updated: 2026-02-19
author: Development Team
tags: [provider, organization, dialog, edit]

関連ドキュメント

- 組織管理メイン画面
- 組織追加ダイアログ
- 組織管理API仕様
- organizationテーブルDDL
- UIポリシー

---

1. 画面レイアウト

1.1. 画面イメージ


組織編集
* 組織CD
ORG001-AIML
* 組織表示名
AIML研究所
* 組織e-mail
lab001@example.com
* 代表システム管理者
システム管理　花子
会社名
アルファテクノロジーズ株式会社
* 契約容量
999
* 契約ユーザー数
999
* 契約プロジェクト数
999
* 二段階認証判定
なし
* みなし進捗使用フラグ
なし
* 状態
アクティブ
キャンセル
保存

2. 概要

2.1. 目的

プロバイダーが既存の組織情報を編集するためのダイアログを定義します。組織表示名、組織e-mail、代表システム管理者、会社名、二段階認証判定、アクティブ状態を変更できます。組織CDは変更不可（主キーのため）。クライアント証明書のダウンロード機能も提供します。

2.2. 利用権限

- **Provider（プロバイダー）**: 使用可能

---

3. ダイアログ構成

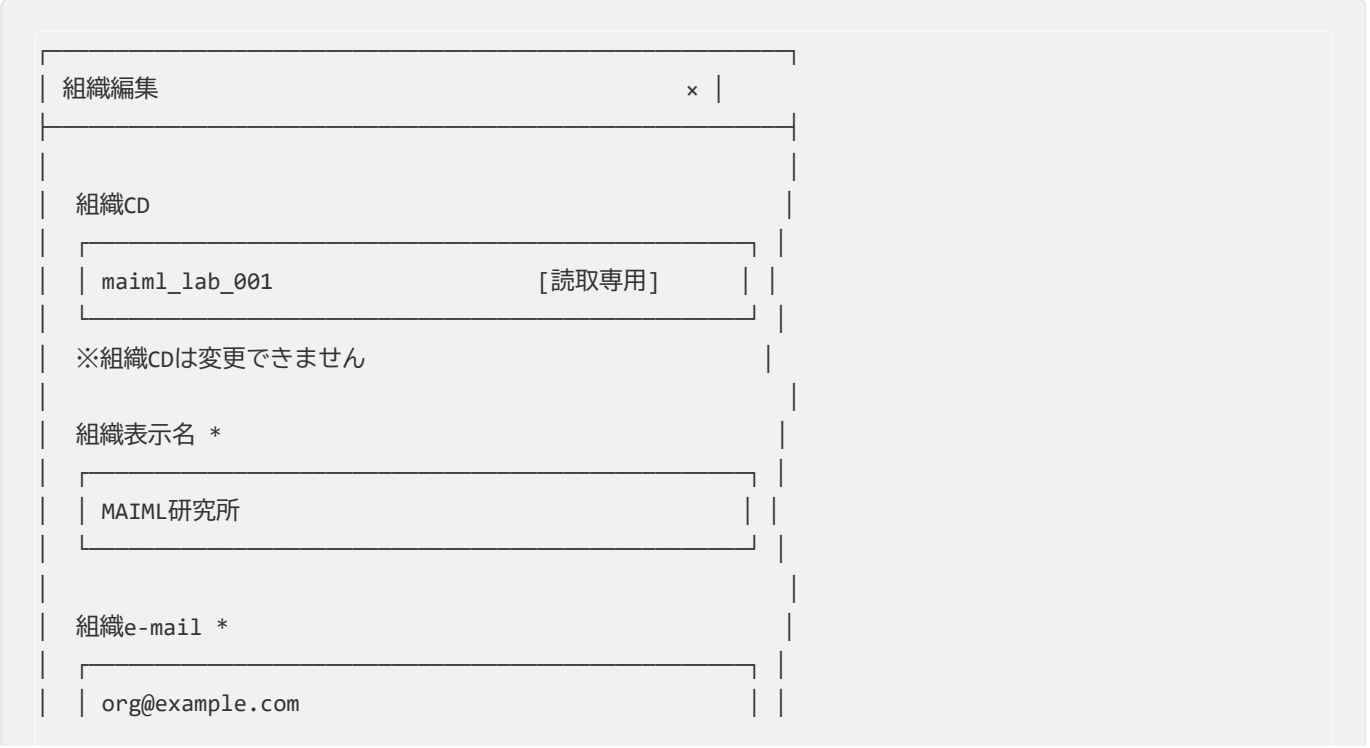
3.1. ダイアログ概要

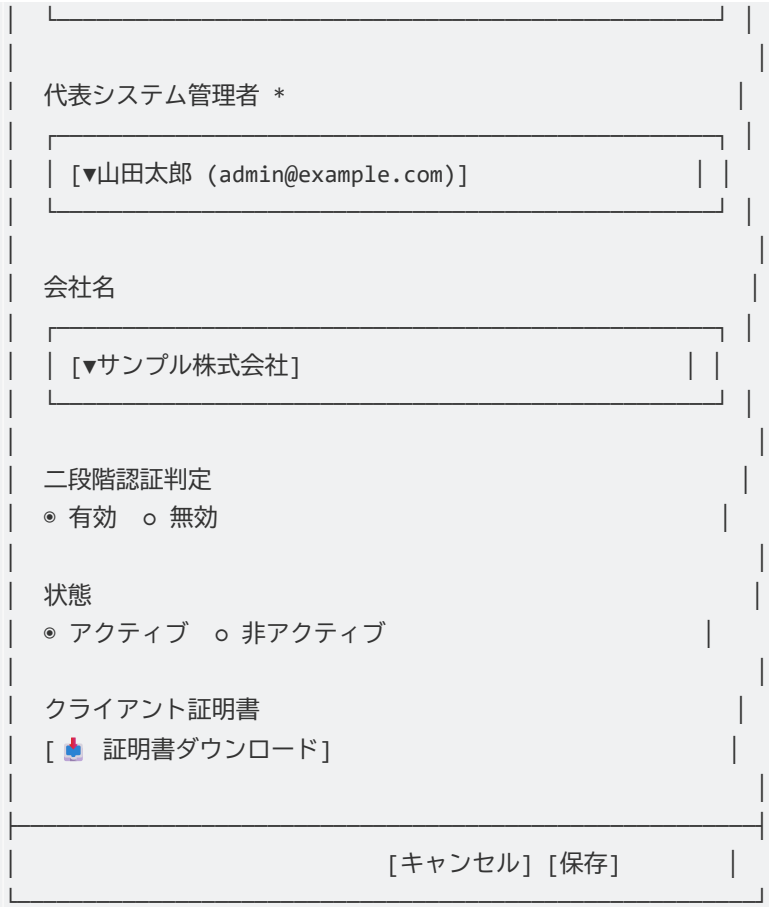
表示トリガー: 組織管理画面の「編集」ボタンクリック

モーダルタイプ: Modal

サイズ: 大（800px）

3.2. レイアウト





4. 入力項目

4.1. 組織CD（表示のみ）

コンポーネント: `<Form.Item>` + `<Input>`

仕様:

項目	値
ラベル	「組織CD」
型	text
読み取り専用	○（disabled）
説明	「※組織CDは変更できません」

実装例:

```
<Form.Item
  label="組織CD"
  name="organizationCd"
  extra="※組織CDは変更できません"
>
  <Input disabled />
</Form.Item>
```

4.2. 組織表示名（必須）

コンポーネント: `<Form.Item>` + `<Input>`

仕様:

項目	値
ラベル	「組織表示名」
プレースホルダー	「組織表示名を入力」
型	text
必須	○
最小文字数	1文字
最大文字数	100文字

バリデーション:

```
{
  required: true,
  message: '組織表示名を入力してください'
},
{
  min: 1,
  max: 100,
  message: '組織表示名は1～100文字で入力してください'
}
```

4.3. 組織e-mail（必須）

コンポーネント: `<Form.Item>` + `<Input>`

仕様:

項目	値
ラベル	「組織e-mail」
プレースホルダー	「organization@example.com」
型	email
必須	○
最大文字数	100文字

バリデーション:

```
{
  required: true,
  message: '組織e-mailを入力してください'
},
{
  type: 'email',
  message: '有効なメールアドレスを入力してください'
},
{
  max: 100,
  message: '組織e-mailは100文字以内で入力してください'
}
```

4.4. 代表システム管理者（必須）

コンポーネント: `<Form.Item>` + `<Select>`

仕様:

項目	値
ラベル	「代表システム管理者」
プレースホルダー	「システム管理者を選択」
型	select
必須	○
データソース	専用API

データ取得:

- API: `GET /api/v1/provider/organization/:organizationId/system-admins`
- レスポンス: `{ userId, userName }` の配列
- 表示: `userName (userId)`
- 値: userId

実装例:

```
<Form.Item
  label="代表システム管理者"
  name="chiefUserId"
  rules={[{ required: true, message: '代表システム管理者を選択してください' }]}
>
  <Select
    placeholder="システム管理者を選択"
    options={systemAdmins.map(admin => ({
      label: `${admin.userName} (${admin.userId})`,
      value: admin.userId
    }))}
  />
</Form.Item>
```

4.5. 会社名（任意）

コンポーネント: `<Form.Item>` + `<Select>`

仕様:

項目	値
ラベル	「会社名」
プレースホルダー	「選択してください」
型	select
必須	×
データソース	corporationテーブル

データ取得:

- API: `GET /api/v1/provider/corporation`

- レスポンス: `{ corporationId, corporationName }` の配列
- 表示: corporationName
- 値: corporationId

4.6. 二段階認証判定（必須）

コンポーネント: `<Form.Item>` + `<Radio.Group>`

仕様:

項目	値
ラベル	「二段階認証判定」
選択肢	有効(1) / 無効(0)
必須	○

実装例:

```
<Form.Item
  label="二段階認証判定"
  name="isEntry"
  rules={[{ required: true, message: '二段階認証判定を選択してください' }]}
>
  <Radio.Group>
    <Radio value={1}>有効</Radio>
    <Radio value={0}>無効</Radio>
  </Radio.Group>
</Form.Item>
```

4.7. 状態（必須）

コンポーネント: `<Form.Item>` + `<Radio.Group>`

仕様:

項目	値
ラベル	「状態」
選択肢	アクティブ / 非アクティブ

必須	○

実装例:

```
<Form.Item
  label="状態"
  name="isActive"
  rules={[{ required: true, message: '状態を選択してください' }]}
>
  <Radio.Group>
    <Radio value={true}>アクティブ</Radio>
    <Radio value={false}>非アクティブ</Radio>
  </Radio.Group>
</Form.Item>
```

4.8. クライアント証明書ダウンロード

コンポーネント: `<Button>`

仕様:

項目	値
ラベル	「証明書ダウンロード」
タイプ	`default`
アイコン	`<DownloadOutlined />`
配置	フォーム内（入力項目の下）

動作:

1. ボタンクリック
2. API呼び出し: `GET /api/v1/provider/organization/:organizationId/certificate`
3. PKCS#12形式（.p12ファイル）をダウンロード
4. ファイル名: `{organizationId}.p12`
5. パスワード: `mmdpf123`（デフォルト）

実装例:

```
<Form.Item label="クライアント証明書">
  <Button
    icon={<DownloadOutlined />}
    onClick={handleDownloadCertificate}
    loading={isDownloading}
  >
    証明書ダウンロード
  </Button>
  <div style={{ marginTop: 8, fontSize: 12, color: '#666' }}>
    ※ブラウザにインポートして使用してください（パスワード: mmdpf123）
  </div>
</Form.Item>
```

---

5. データの初期化

5.1. ダイアログ表示時

編集対象の組織データとシステム管理者一覧を取得してフォームに設定:

```
useEffect(() => {
  if (open && organizationId) {
    fetchOrganizationData(organizationId);
    fetchSystemAdmins(organizationId);
    fetchCorporations();
  }
}, [open, organizationId]);

const fetchOrganizationData = async (id: string) => {
  try {
    const response = await apiClient.get(
      `/api/v1/provider/organization/${id}`
    );

    if (response.data.success) {
      const data = response.data.data;
      form.setFieldsValue({
        organizationCd: data.organizationCd,
        organizationDisplayName: data.organizationDisplayName,
        organizationEmail: data.organizationEmail,
        chiefUserId: data.chiefUserId,
        corporationId: data.corporationId,
```

```
        isEntry: data.isEntry,
        isActive: data.isActive
      });
    }
  } catch (error) {
    message.error('組織情報の取得に失敗しました');
    onClose();
  }
};

const fetchSystemAdmins = async (id: string) => {
  try {
    const response = await apiClient.get(
      `/api/v1/provider/organization/${id}/system-admins`
    );

    if (response.data.success) {
      setSystemAdmins(response.data.data);
    }
  } catch (error) {
    console.error('Failed to fetch system admins:', error);
  }
};
```

6. APIリクエスト仕様

6.1. データ取得エンドポイント

組織詳細取得: `GET /api/v1/provider/organization/:organizationId`

システム管理者一覧取得: `GET /api/v1/provider/organization/:organizationId/system-admins`

会社一覧取得: `GET /api/v1/provider/corporation`

6.2. 更新エンドポイント

`PUT /api/v1/provider/organization/:organizationId`

リクエストヘッダー:

```
Content-Type: application/json
Authorization: Bearer {jwt_token}
```

リクエストボディ:

```
{
  "organizationDisplayName": "MAIML研究所（更新）",
  "organizationEmail": "updated@example.com",
  "chiefUserId": "admin2@example.com",
  "corporationId": "corp-uuid-002",
  "isEntry": 1,
  "isActive": true
}
```

レスポンス（成功時）:

```
{
  "success": true,
  "data": {
    "organizationId": "550e8400-e29b-41d4-a716-446655440000",
    "organizationCd": "maiml_lab_001",
    "organizationDisplayName": "MAIML研究所（更新）",
    "organizationEmail": "updated@example.com",
    "chiefUserId": "admin2@example.com",
    "chiefUserName": "佐藤花子",
    "corporationName": "新会社名",
    "isEntry": 1,
    "isActive": true,
    "updatedAt": "20260219132000"
  },
  "meta": {
    "timestamp": "2026-02-19T13:20:00Z",
    "requestId": "req_abc123"
  }
}
```

6.3. 証明書ダウンロードエンドポイント

`GET /api/v1/provider/organization/:organizationId/certificate`

レスポンス: PKCS#12形式のバイナリデータ（.p12ファイル）

---

7. エラーメッセージ

7.1. バリデーションエラー

エラー条件	メッセージ
組織表示名が空	「組織表示名を入力してください」
組織表示名が長すぎる	「組織表示名は1〜100文字で入力してください」
組織e-mailが空	「組織e-mailを入力してください」
組織e-mailが無効	「有効なメールアドレスを入力してください」
組織e-mailが長すぎる	「組織e-mailは100文字以内で入力してください」
代表システム管理者が未選択	「代表システム管理者を選択してください」
状態が未選択	「状態を選択してください」

7.2. APIエラー

エラーコード	メッセージ
`ORGANIZATION_NOT_FOUND`	「組織が見つかりません」
`VALIDATION_ERROR`	「入力内容に誤りがあります」
`UNAUTHORIZED`	「この操作を実行する権限がありません」
`CERTIFICATE_NOT_FOUND`	「証明書が見つかりません」
`INTERNAL_SERVER_ERROR`	「サーバーエラーが発生しました。しばらく経ってから再試行してください」

---

8. アクションボタン

8.1. キャンセルボタン

動作:

1. 入力内容を破棄
2. ダイアログを閉じる
3. 親画面に戻る

8.2. 保存ボタン

動作:

1. フォームバリデーション実行
2. 変更があるか確認
3. API呼び出し: `PUT /api/v1/provider/organization/:organizationId`
4. 成功時:
    - 成功メッセージ表示（Toast）
    - ダイアログを閉じる
    - 親画面の一覧を再取得
5. 失敗時: エラーメッセージ表示

---

9. テスト要件

9.1. 単体テスト

テストケース

- データ取得機能のテスト
- システム管理者一覧取得のテスト
- バリデーション機能のテスト
- フォーム送信処理のテスト
- 証明書ダウンロード機能のテスト

9.2. 統合テスト

テストシナリオ

1. 正常な組織編集
2. 組織表示名のみ変更
3. 代表システム管理者のみ変更
4. 状態のみ変更
5. 変更なしで保存
6. 存在しない組織の編集
7. 非アクティブ化の確認ダイアログ
8. 証明書ダウンロード成功
9. 証明書ダウンロード失敗（証明書なし）

---

10. アクセシビリティ

10.1. キーボード操作

- Tab: 次の入力項目へフォーカス移動
- Shift + Tab: 前の入力項目へフォーカス移動
- Enter: フォーム送信
- Escape: ダイアログを閉じる

10.2. スクリーンリーダー対応

- 読み取り専用フィールドに `aria-readonly="true"` を設定
- エラーメッセージを `role="alert"` で通知
- 必須項目を `aria-required="true"` で明示

11. 変更履歴

バージョン	日付	変更内容	担当者

1.0.0	2026-02-19	初版作成	Development Team